MouseMine Build — Job Aid

Quick reference for accessing the MouseMine container and running a build manually.

---

1. Connect

```
# SSH to AllianceMineDev
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.60.197

# Enter the MouseMine container
docker exec -it mousemine bash
```

You are now `root` inside the container with Ubuntu 20.04, Java 11, Ant, and PostgreSQL.

---

2. Check the Environment

```
# Verify PostgreSQL is running
pg_isready                          # Should say "accepting connections"

# Verify Java
java -version                       # OpenJDK 11

# Verify Ant
ant -version                        # Apache Ant 1.10.7

# Check properties file exists
cat /root/.intermine/mousemine.properties | head -5
```

If the properties file is missing, see section 7 below.

---

3. Transfer Data In

Data can be transferred into the container two ways:

Option A: Via the mounted `/data` directory (recommended)

The `/data` directory inside the container maps to the host at:

`/home/ec2-user/agr_mousemine/mousemine_data/`

From your local machine, copy files to the host mount:

```
# From your machine → host
scp -i ~/.ssh/AGR-ssl3.pem myfile.gff \
  ec2-user@172.31.60.197:~/agr_mousemine/mousemine_data/

# The file appears inside the container at /data/myfile.gff
```

Option B: docker cp (for individual files)

```
# From host → container
docker cp /path/to/file mousemine:/desired/path/inside/container

# Example: copy a GFF file
docker cp ~/my_data/genes.gff mousemine:/mousemine_etl/output/mgi-gff/latest/
```

Option C: Download directly inside the container

```
# Inside the container
curl -o /data/myfile.gz https://example.com/myfile.gz
wget https://example.com/myfile.gz -P /data/
```

---

4. Run the ETL (Data Extraction)

The ETL downloads data from MGI, ontology sources, UniProt, InterPro, etc.

```
cd /mousemine_etl

# Run all data sources
python3 bin/refresh.py

# Run a specific source only
python3 bin/refresh.py -s mgi-base
python3 bin/refresh.py -s go
python3 bin/refresh.py -s uniprot
```

The ETL writes to `/mousemine_etl/output/{source}/latest/`.

Important: `project.xml` expects data at `/var/lib/jenkins/etl_build/etl/output`. Create a symlink if it doesn't exist:

```
mkdir -p /var/lib/jenkins/etl_build/etl
ln -sf /mousemine_etl/output /var/lib/jenkins/etl_build/etl/output
```

Verify data is accessible:

```
ls /var/lib/jenkins/etl_build/etl/output/so/latest/
# Should show: SequenceOntology.obo
```

---

5. Run the Build

All commands from `/mousemine`:

```
cd /mousemine
```

Fresh build (from scratch)

```
# Step 1: Build database schema
ant -Drelease=1.8 -v build-db

# Step 2: Integrate all data sources (hours)
/intermine/project_build -b -E UTF8 localhost /data/dump

# Step 3: Post-processing (indexes, references)
ant -v postprocess

# Step 4: Build user profile database
ant -v build-userdb

# Step 5: Build WAR file
ant -v war
```

Resume after failure

```
# Load last checkpoint and continue
/intermine/project_build -l -E UTF8 localhost /data/dump

# Then finish remaining steps
ant -v postprocess
ant -v build-userdb
ant -v war
```

Run a single source manually

```
cd /mousemine
ant -Dsource=mgi-base -v integrate
ant -Dsource=go -v integrate
```

---

6. Monitor Progress

```
# Watch the build log
tail -f /data/dump/pbuild.log

# Check database size (local PostgreSQL)
su - postgres -c "psql -c \"SELECT pg_size_pretty(pg_database_size('mousemine'))\""

# Check database size (RDS)
PGPASSWORD='...' psql -h intermine-postgres.cmnnhlso7wdi.us-east-1.rds.amazonaws.com \
  -U postgres -d postgres -c "SELECT pg_size_pretty(pg_database_size('mousemine_db'))"

# Check what source is running
grep 'starting command' /data/dump/pbuild.log | tail -1

# Check container resource usage (from host, not inside container)
# docker stats mousemine
```

---

7. Properties File Setup

If `/root/.intermine/mousemine.properties` doesn't exist, create it:

Using local PostgreSQL (already running in container)

```
mkdir -p /root/.intermine
cat > /root/.intermine/mousemine.properties << 'EOF'
db.production.datasource.serverName=localhost
db.production.datasource.databaseName=mousemine
db.production.datasource.user=mousemine
db.production.datasource.password=mousemine

db.common-tgt-items.datasource.serverName=localhost
db.common-tgt-items.datasource.databaseName=items
db.common-tgt-items.datasource.user=mousemine
db.common-tgt-items.datasource.password=mousemine

db.userprofile-production.datasource.serverName=localhost
db.userprofile-production.datasource.databaseName=userprofile
db.userprofile-production.datasource.user=mousemine
```

```
db.userprofile-production.datasource.password=mousemine

project.title=MouseMine
project.releaseVersion=1.8
webapp.deploy.url=http://localhost:8080
webapp.baseurl=http://localhost:8080
webapp.path=mousemine
webapp.manager=manager
webapp.password=manager
superuser.account=superuser@mail_account
index.solrurl=http://localhost:8983/solr/mousemine-search
autocomplete.solrurl=http://localhost:8983/solr/mousemine-autocomplete
EOF
```

Using RDS

Same as above but replace `localhost` with the RDS endpoint and credentials. Ask the team lead for the RDS password — **do not commit it to any file**.

---

8. Container Cheat Sheet

Task	Command
Enter container	`docker exec -it mousemine bash`
Copy file in	`docker cp file mousemine:/path`
Copy file out	`docker cp mousemine:/path file`
View logs	`docker logs mousemine`
Restart container	`docker restart mousemine`
Check health	`docker inspect mousemine --format '{{.State.Health.Status}}'`
Stop container	`docker stop mousemine` (stops all services)
Start container	`docker start mousemine`

---

9. Directory Map

```
/mousemine/                          # Mine project
  ├── project.xml                    # Data source definitions
  ├── build.xml                      # Ant build file
  ├── dbmodel/                       # Database model
  └── webapp/                        # Web application

/intermine/                          # InterMine core (same as /mousemine symlink)
  └── project_build                  # Perl build script

/mousemine_bio_sources_mgi/          # MGI-specific data loaders
  ├── mgi-base/                      # Core MGI data
  ├── mgi-fasta/                     # FASTA sequences
  ├── mgi-gff/                       # GFF features
  └── ...                            # Other MGI sources

/mousemine_etl/                      # ETL data extraction
  ├── bin/refresh.py                 # Main ETL driver
```

```
  ├── bin/config.cfg                # Source URLs and commands
  ├── bin/libdump/                  # Python data dumpers
  └── output/                      # Downloaded data (→ symlink to /var/lib/jenkins/...)

/data/                              # Mounted from host (persistent)
  └── dump.*                        # Build checkpoint files

/root/.intermine/
  └── mousemine.properties          # Database connection config
```

---

10. Common Issues

Problem	Solution
`mousemine.properties not found`	Create it — see section 7
`No such file: /var/lib/jenkins/etl_build/...`	Create symlink: `ln -sf /mousemine_etl/output /var/lib/jenkins/etl_build/etl/output`
`SQL_ASCII encoding error`	Use `-E UTF8` with `project_build`
`Connection refused` to RDS	Check VPN is connected, check security groups
ETL source fails	Check `bin/config.cfg` for URL changes, run with `-s <source>` to retry one
Out of memory	`export ANT_OPTS="-Xmx8g"` before running ant
PostgreSQL not running	`su - postgres -c 'pg_ctl start -D /var/lib/postgresql/12/main'`